数字图像处理

实验原理总册

2026 学年 · 计算机科学与工程学院

1. 实验一 直方图均衡化与规定化
2. 实验二 变换与伽马校正
3. 实验三 噪声去除
4. 实验四 空间域锐化
5. 实验五 频率域低通与高通滤波
6. 实验六 图像复原
7. 实验七 人脸皮肤颜色分层

本册汇总七次实验的底层原理、公式推导与答辩问答

公式经 LaTeX 渲染 · 可直接打印

实验一：直方图均衡化 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**把一张"灰蒙蒙"的照片，变得"黑白分明"。**

你拿到一张偏暗、低对比度的图（比如雾天拍的），像素灰度值全挤在一个小范围内（比如都集中在50~100）。直方图均衡化做的事，就是**把这些挤在一起的灰度值"摊开"，均匀铺满 0~255 的整个范围**，这样暗的更暗、亮的更亮，对比度就出来了。

---

2. 先搞懂：什么是直方图？

灰度图每个像素是一个 0~255 的数（0=纯黑，255=纯白）。

直方图 = 数出每个灰度值出现了多少次。

$$h(r_k) = n_k$$

其中： - r_k = 第 k 个灰度级（$k = 0, 1, ..., 255$） - n_k = 灰度值等于 r_k 的像素个数

大白话：直方图就是"**这张图里，每个亮度档位各有多少个像素**"的统计柱状图。

答辩常问：**直方图横轴纵轴是什么？**

> 横轴 = 灰度级（0~255），纵轴 = 该灰度级出现的像素个数（或频率）。

---

3. 归一化直方图（概率）

光看"个数"不好比较（大图小图总数不同），所以除以总像素数 N：

$$p(r_k) = \frac{n_k}{N}, \quad \sum_{k=0}^{L-1} p(r_k) = 1$$

这就变成了"**每个亮度档位占全图的比例**"，本质上是一个**概率分布**。

答辩常问：**为什么要归一化？**

> 归一化后就成了概率分布，累积起来才能得到"累积分布函数 CDF"，而均衡化的映射公式就是基于 CDF 推导的；同时归一化让不同尺寸的图可以互相比较。

---

4. 核心：均衡化为什么用 CDF（累积分布函数）？

这是**最最关键的原理**，老师最爱问。

4.1 直观理解

均衡化的目标是：让输出的灰度**尽可能均匀分布**。

数学上有个经典结论（随机变量变换）：

> 如果把一个随机变量 r 经过**它自己的累积分布函数** $T(r)$ 变换，得到的新变量 $s = T(r)$ 就**服从均匀分布**。

这就是直方图均衡化的数学根基。

4.2 累积分布函数 CDF

$$CDF(r_k) = \sum_{j=0}^{k} p(r_j)$$

含义：**灰度值 ⩽ r_k 的像素占全图的比例**。

大白话：CDF 就是"从最黑开始，一路累加到当前亮度，一共攒了多少比例的像素"。

4.3 均衡化映射公式（背下来）

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p(r_j) = (L-1)\cdot CDF(r_k)$$

其中 $L = 256$（灰度级总数），所以 $L - 1 = 255$。

含义：把原来灰度值 r_k，映射成新灰度值 s_k —— 映射规则就是"**用它的累积概率乘以 255**"。

4.4 为什么这个公式能"摊开"?

- 如果一堆像素挤在某个窄范围（密度大），那这段 CDF 增长很快 → 映射后跨度大 → **被拉开了**
- 如果某段像素很少（密度小），CDF 增长慢 → 映射后跨度小 → **被压缩了**

正好是"**密的拉开、疏的压缩**"，结果就均匀了。

答辩常问：**为什么均衡化公式是 CDF 乘 255，而不是别的?**

> 因为概率积分变换定理：任意连续随机变量经过自身 CDF 变换后服从 [0,1] 均匀分布。乘 $(L-1)$ 只是把 [0,1] 拉伸到灰度范围 [0,255]。这是让输出直方图尽可能平（均匀）的最优单调映射。

答辩常问：**均衡化有什么缺点?**

> 1. 是**全局**方法，对局部暗/亮区域照顾不到（引出后面的自适应均衡化 CLAHE）；
> 2. 会**增强噪声**（原本平坦区域的微小噪声被一并拉伸）；
> 3. 可能产生**灰度级合并**（"简并"），输出灰度级数变少；
> 4. 结果**不可控**，只能得到近似均匀，不能指定想要的分布形状（这就引出了直方图规定化）。

5. 亮度/对比度变换（最基础的线性变换）

在均衡化之前，还有个更基础的操作——**点运算（逐像素独立变换）**：

$$s = \alpha \cdot r + \beta$$

- α（alpha）= **对比度**增益：$\alpha > 1$ 拉伸对比度，$\alpha < 1$ 压缩
- β（beta）= **亮度**偏移：$\beta > 0$ 变亮，$\beta < 0$ 变暗

大白话：$s = ar + \beta$ 就是"**乘个倍数再加减个常数**"，跟初中直线方程 $y = kx + b$ 一模一样。

答辩常问：**亮度变换和对比度变换区别？**

> 亮度（β）是整体平移，直方图**左右移动**，形状不变；对比度（α）是缩放，直方图**被拉伸或压缩**。

---

6. 直方图规定化（匹配）——升级版

均衡化只能得到"均匀"分布，**规定化（Matching / Specification）** 能做到：**我想要什么形状的直方图，就给你变成什么形状。**

步骤（三步走）： 1. 对原图做均衡化：$s = T(r)$，得到均匀分布 2. 对**目标**直方图也做均衡化：$v = G(z)$，也得到均匀分布 3. 因为两者都均匀，所以令 $s = v$，反解出 $z = G^{-1}(s)$

$$\boxed{z = G^{-1}(T(r))}$$

大白话：**原图 →(均衡化)→ 均匀 →(目标分布的逆均衡化)→ 目标形状**。均衡化在这里当"桥梁"。

答辩常问：**规定化和均衡化什么关系？**

> 均衡化是规定化的**特例**——当目标直方图取均匀分布时，规定化就退化成均衡化。规定化更灵活，可以指定想要的输出分布。

答辩常问：**规定化一定能精确匹配目标吗？**

> 离散情况下**不能精确匹配**，因为灰度级是离散的（只有 256 级），映射必须取整，会有"简并"和近似，只能做到**最接近**。连续理论上是精确的。

---

7. 本实验代码实现对照

理论	代码位置	说明
线性变换 $s = ar + \beta$	`adjust_brightness_contrast()`	`cv2.convertScaleAbs`
直方图统计 $h(r_k)$	`cv2.calcHist`	统计各灰度级频数
均衡化 $s_k = 255 \cdot CDF(r_k)$	`cv2.equalizeHist`	OpenCV 内置
规定化 $z = G^{-1}(T(r))$	自定义映射表	通过 CDF 匹配实现
自动读标准图	`load_image()`	优先 `data/standard/`

运行：

```
python hist_equalization.py
python hist_equalization.py ../../../../data/standard/lena.jpg
```

---

8. 答辩高频问题速答

Q：直方图均衡化是全局还是局部？

> 本实验是**全局**（整幅图一个映射函数）。局部版本叫 CLAHE（限制对比度自适应直方图均衡化），把图分块各自均衡化再插值。

Q：对彩色图怎么做均衡化？

> **不要直接对 RGB 三通道分别均衡化**，会严重偏色（因为改变了颜色通道间的比例关系）。正确做法是转到 HSV/YCrCb **空间，只对亮度分量 V 或 Y 做均衡化**，保持色度 H/S 或 Cr/Cb 不变，再转回 RGB。

Q：均衡化后图像一定更好看吗？

> 不一定。如果原图直方图已经很均匀，均衡化**几乎没变化甚至变差**；对噪声大的图会放大噪声。它只对"**直方图集中在窄范围**"的低对比度图效果显著。

Q：为什么叫"点运算"？

> 因为变换**只取决于单个像素自身的灰度值**，与邻域像素无关，逐点独立处理，所以也叫灰度变换/点处理。

Q：直方图均衡化的数学本质？

> 概率积分变换：输出灰度 = 输入灰度的累积分布函数 × 最大灰度级。让输出服从（近似）均匀分布。

实验二：伽马校正 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**用一条"弯曲的曲线"重新分配亮度，把太暗的细节拉出来，或把太亮的压下去。**

实验一是"均匀摊开"（强制均匀），实验二则是"**按需弯曲**"——你可以精确控制：想提亮暗部（gamma<1）还是压暗整体（gamma>1）。

典型场景：显示器/相机有非线性响应（伽马特性），拍出来偏暗，需要 gamma 校正还原真实亮度。

---

2. 核心公式：幂律变换（背下来）

$$s = c \cdot r^{\gamma}$$

其中： - r = 输入灰度，**归一化到 [0,1]**（所以要先除以 255） - s = 输出灰度，也在 [0,1] - c = 常数增益，通常取 $c = 1$ - γ（gamma）= **伽马值**，决定曲线的"弯曲方向"

代码实现（注意归一化）：

```
table = np.array([c * ((i / 255.0) ** gamma) * 255 for i in range(256)])
```

先把 `i/255.0` 归一化到 [0,1]，做幂运算，再乘回 255。

答辩常问：**为什么必须先除以 255 归一化？**

> 因为幂律变换定义在 [0,1] 区间。如果直接对 0~255 做幂运算，数值会爆炸（比如 $255^{2.2}$ 是个巨大数字），且结果完全错误。归一化后保证输入输出都在 [0,1]，乘回 255 才回到 0~255 灰度范围。

---

3. Gamma 不同取值的效果（最重要）

因为 $r \in [0,1]$，所以：

gamma	曲线形状	效果	适用
$\gamma < 1$	曲线**上凸**（在对角线上方）	**提亮**，暗部被拉伸	图像太暗，想看清暗部细节
$\gamma = 1$	直线 $s = r$	**原图不变**	恒等变换
$\gamma > 1$	曲线**下凹**（在对角线下方）	**变暗**，亮部被拉伸	图像太亮/过曝

为什么 $\gamma < 1$ 会提亮？ 举个栗子，$r = 0.25$（较暗）：- $\gamma = 0.5$: $s = 0.25^{0.5} = 0.5$ → 从 0.25 提到 0.5，**变亮了** - $\gamma = 2.0$: $s = 0.25^2 = 0.0625$ → 从 0.25 压到 0.06，**变暗了**

因为在 (0,1) 区间内，**越小的数，开方（$\gamma < 1$）后相对变大；平方（$\gamma > 1$）后相对变小。**

本实验测试的 gamma 值：`[0.4, 0.6, 1.0, 1.5, 2.2, 3.0]` - 0.4、0.6：明显提亮（暗部细节浮现） - 2.2、3.0：明显变暗（2.2 是显示器标准伽马！）

答辩常问：gamma=2.2 **有什么特殊?**

> 2.2 是**典型显示器的伽马值**。显示器亮度与电压呈 ≈ 2.2 次幂关系，所以相机/图像存储时常做 1/2.2 的"伽马编码"，显示时显示器自动 2.2 解码，正好抵消，还原线性亮度。这就是"伽马校正"名字的由来。

---

4. 为什么幂律变换能"选择性拉伸"？（数学本质）

对 $s = r^\gamma$ 求导，看**曲线斜率**（斜率 = 变换的"放大倍数"）：

$$\frac{ds}{dr} = \gamma \cdot r^{\gamma-1}$$

- 当 $\gamma < 1$：在 r 小（暗部）的地方，$\gamma r^{\gamma-1}$ **很大** → 暗部被放大拉伸 → 提亮暗部
- 当 $\gamma > 1$：在 r 大（亮部）的地方，斜率大 → 亮部被拉伸

大白话：**曲线越陡的地方，输入的小变化会导致输出的大变化 → 那一段就被"拉开"了，细节更丰富。**

答辩常问：**伽马变换和直方图均衡化有什么区别?**

> 1. 伽马是**固定参数曲线**，形状由 γ 决定，是"预设"的变换；均衡化是**数据驱动**，映射函数由图像自身直方图算出，自适应；
> 2. 伽马只能做单调的幂函数变形，不能让直方图变平；均衡化目标是让直方图均匀；
> 3. 伽马校正常用于**校正设备的非线性响应**（有物理意义），均衡化纯粹是**增强对比度**。

---

5. 另外两个变换：对数 & 反色

5.1 对数变换（扩展暗部）

$$s = c \cdot \log(1 + r)$$

（r 归一化到 [0,1]，$\log(1 + r)$ 保证 $r = 0$ 时 $s = 0$）

特点：把**窄范围的低灰度**映射到**宽范围输出**，强烈扩展暗部，压缩亮部。

典型应用：**傅里叶频谱显示**！频谱的动态范围极大（中心值可能上万，边缘接近 0），直接显示几乎全黑，必须取 log 压缩才能看清结构。这正好解释了实验五为什么频谱图要用对数变换。

答辩常问：**为什么频谱图要取** log?

> 因为频谱的**动态范围太大**（直流分量可能比高频分量大几个数量级），线性显示时只有中心一个亮点，其余全黑。取对数后把大值压缩、小值提升，才能看到完整的频率分布结构。

5.2 反色变换（负片）

$$s = (L-1) - r = 255 - r$$

把黑白颠倒，用于医学影像（X 光片看负片有时更易发现病灶）。就是"**用 255 减去原值**"。

---

6. 本实验代码实现对照

理论	代码位置	说明
幂律 $s = c \cdot r^{\gamma}$	`gamma_transform()`	LUT 查表加速
对数 $s = c\log(1+r)$	`log_transform()`	扩展暗部
反色 $s = 255 - r$	`negative_transform()`	负片
gamma 曲线绘制	`plot_gamma_curves()`	可视化不同 γ

运行：

```
python gamma_correction.py
python gamma_correction.py ../../../../data/standard/lena.jpg
```

---

7. 答辩高频问题速答

Q：为什么用查找表 LUT 实现而不是逐像素算？

> 灰度只有 256 个可能值，变换是**逐点**的（与邻域无关），所以只需预先算好 256 个值的映射表，然后用 `table[img]` 一次索引完成整图变换。**极大加速**，避免百万次重复幂运算。

Q：伽马变换是点运算吗？

> 是。输出只取决于**单个像素自身的灰度值**，与周围像素无关。

Q：gamma 校正能增强对比度吗？

> 能，但方式不同。它是**非线性地重新分配**灰度：暗部或亮部被拉伸，中间调被压缩。相比线性拉伸（实验一的 $ar+\beta$），伽马更灵活，能"有侧重"地增强。

Q：s = c · r^γ 中 c 的作用？

整体缩放增益。通常取 $c=1$；若 $c>1$ 整体变亮，$c<1$ 整体变暗。但因为归一化要求输入输出在 $[0,1]$，一般取 $c=1$ 或 $c=1/\max(r^{\gamma})$ 做归一化。

Q：对数变换和伽马变换（γ<1）都能提亮暗部，区别?

对数变换对**极暗部分（接近 0）** 的提升更剧烈（log 在 0 附近斜率无穷大），压缩高亮更狠，常用于**频谱显示**；伽马 $\gamma<1$ 的曲线更"温和"可控，通过 γ 连续调节，常用于**显示器校正**。两者都是非线性单调变换。

实验三：图像去噪 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**给图像"加噪声"再"想办法去掉"，比较哪种去噪方法最管用。**

先人工给干净图加两类噪声（高斯噪声、椒盐噪声），然后用不同的滤波器去噪，最后用 PSNR 打分，看谁恢复得最好。

---

2. 两类噪声：本质完全不同（超重要）

2.1 高斯噪声

每个像素都叠加一个**服从高斯分布（正态分布）的随机扰动**：

$$g(x,y) = f(x,y) + n(x,y), \quad n \sim \mathrm{N}(\mu, \sigma^2)$$

其中概率密度：

$$p(n) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(n-\mu)^2}{2\sigma^2}}$$

- μ = 均值，通常取 0
- σ = 标准差，**决定噪声强度**（σ 越大越噪）

特点：**每个像素都"轻微抖动"**，整幅图蒙上一层细腻的颗粒感（像老电视雪花）。

2.2 椒盐噪声（脉冲噪声）

随机把**部分像素**直接变成**纯白（255，盐）或纯黑（0，椒）**：

$$g(x,y) = \begin{cases} 0 & \text{以概率 } p_{pepper}\text{（椒，黑点）} \\ 255 & \text{以概率 } p_{salt}\text{（盐，白点）} \\ f(x,y) & \text{以概率 } 1 - p_{salt} - p_{pepper} \end{cases}$$

特点：**少数像素被"极端污染"**，其余完好。视觉上是随机的黑白麻点。

代码实现：`amount` = 噪声总比例，`salt_vs_pepper` = 盐在其中的占比，用 `np.random.choice` 随机选位置置 0 或 255。

答辩常问：**高斯噪声和椒盐噪声的根本区别？**

> 高斯噪声是**加性**的、每个像素都有、幅度小且连续（服从正态分布）；椒盐噪声是**脉冲**的、只有部分像素、幅度极端（非 0 即 255）。这个区别直接决定了：高斯噪声适合**均值类平滑**滤除，椒盐噪声必须用**中值类（排序统计）**滤除。

3. PSNR：怎么评价去噪好坏？（必考）

$$MSE = \frac{1}{MN} \sum_{x,y} [f(x,y) - \hat{f}(x,y)]^2$$

$$PSNR = 10 \cdot \log_{10}(\frac{MAX^2}{MSE}) = 10 \cdot \log_{10}(\frac{255^2}{MSE})$$

- f = 原始干净图，$\hat{f}$ = 去噪后的图
- MSE = 均方误差，越小越好
- $MAX = 255$（8 位图最大值）
- PSNR **越大越好**，单位 dB。一般认为 > 30dB 质量较好，> 40dB 非常好

为什么用对数？ 因为 MSE 的动态范围太大（可能从 0.01 到 10000），取对数压缩到 dB 尺度，方便比较。

答辩常问：PSNR **有什么局限？**

> PSNR 只统计**像素级的数值误差**，完全不考虑**人眼视觉感受**（结构、纹理、边缘）。有时 PSNR 高的图人眼看着反而更糊（因为过度平滑）。更先进的指标有 SSIM（结构相似性），考虑了亮度、对比度、结构三方面。

答辩常问：**为什么评价去噪需要"原始干净图"？**

> 因为 PSNR 是**有参考（full-reference）**指标，必须拿去噪结果和真值逐像素比对。实际应用中往往没有真值，只能用无参考指标（如自然场景统计、盲评价）。

---

4. 各类滤波器的原理（核心）

4.1 均值滤波 / 算术均值（线性）

$$\hat{f}(x,y) = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s,t)$$

用邻域窗口内所有像素的**平均值**替代中心像素。

- 效果：**平滑**高斯噪声有效（因为高斯噪声零均值，平均会互相抵消）
- 缺点：**把边缘也模糊了**（边缘像素也被平均掉了）
- 对椒盐噪声**效果差**（极端值会污染平均值）

4.2 中值滤波（非线性 · 排序统计）—— 椒盐噪声的克星

$$\hat{f}(x,y) = \underset{(s,t) \in S_{xy}}{\text{median}}\{g(s,t)\}$$

把窗口内所有像素**排序，取中间那个值**。

为什么能去椒盐？ 因为椒盐噪声是极端的 0 或 255，排序后排在**两端**，**中位数**几乎不受影响！而中值本身取自窗口中某个真实像素，所以**能保留边缘**。

为什么对高斯噪声一般？ 高斯噪声每个像素都轻微偏离，中值只是"选一个"，没有平均的"抵消"效应，去噪能力不如均值。

本实验实测：椒盐噪声下中值滤波 PSNR 高达 45.74 dB，远超其他方法 —— 这就是原因。

答辩常问：**中值滤波为什么能去椒盐噪声又保边？**

> 1. 去椒盐：极值排序后在两端，中位数不受污染；
> 2. 保边：输出值是窗口中**真实存在的某个像素值**，不会造出不存在的中间灰度，跨越边缘时不会被"平均"成过渡值。

4.3 最大值 / 最小值滤波（形态学）

- **最大值滤波** $\hat{f} = \max\{g\}$：用窗口内最大值 → 吃掉**暗点（椒）**，用 `cv2.dilate`（膨胀）实现
- **最小值滤波** $\hat{f} = \min\{g\}$：用窗口内最小值 → 吃掉**亮点（盐）**，用 `cv2.erode`（腐蚀）实现

原理：**膨胀取局部最大 → 暗的被周围亮的覆盖；腐蚀取局部最小 → 亮的被周围暗的覆盖。**

答辩常问：**最大值/最小值滤波分别对哪种噪声有效？**

> 最大值（膨胀）去**椒（黑）噪声**；最小值（腐蚀）去**盐（白）噪声**。注意方向：想要"变亮"用膨胀，"变暗"用腐蚀。

4.4 中点滤波

$$\hat{f} = \frac{1}{2}\left[\max\{g\} + \min\{g\}\right]$$

取窗口内最大值与最小值的**算术平均**。对**高斯噪声和均匀噪声**有一定效果（因为随机噪声的最大最小值围绕真值对称），但对椒盐噪声无能为力（最大最小值恰好就是噪声本身）。

4.5 修正阿尔法均值滤波（本实验最优的混合噪声方案）

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(s,t)\in S_{xy}-d} g_r(s, t)$$

做法：窗口内排序，**去掉最大的 $d/2$ 个和最小的 $d/2$ 个**，剩下的求平均。

为什么最强？ 它**结合了中值滤波（剔除极端值）和均值滤波（平均抵消随机噪声）的优点**： - 去掉两端 → 消除椒盐脉冲 - 剩下求平均 → 平滑高斯噪声

所以它对**混合噪声**（既有高斯又有椒盐）效果最好。本实验混合噪声下 31.41 dB，为最优。

答辩常问：**修正阿尔法均值滤波为什么适合混合噪声？**

> 因为它"两头都占"：排序截尾去除脉冲（椒盐），对剩余像素平均抑制随机噪声（高斯）。单一的中值或均值只能对付其中一类。

---

5. 线性 vs 非线性滤波（分类，老师爱问）

	线性滤波（均值、高斯）	非线性滤波（中值、最大/最小、阿尔法）
运算	加权求和（卷积）	排序、取极值
对高斯噪声	好	一般
对椒盐噪声	差	极好
保边缘	差（模糊）	好
可否用卷积实现	可以	不可以

---

6. 本实验代码对照

理论	代码位置	实现
高斯噪声	`add_gaussian_noise()`	`np.random.normal`
椒盐噪声	`add_salt_pepper_noise()`	随机位置置 0/255
PSNR	`psnr()`	$10\log_{10}(255^2/MSE)$
中值滤波	`median_filter()`	`cv2.medianBlur`
最大值（膨胀）	`max_filter()`	`cv2.dilate`
最小值（腐蚀）	`min_filter()`	`cv2.erode`
中点滤波	`midpoint_filter()`	(max+min)/2
修正阿尔法均值	主流程	排序截尾平均

运行：

```
python denoising.py
python denoising.py ../../../../data/standard/baboon.jpg
```

---

7. 答辩高频问题速答

Q：为什么均值滤波去高斯噪声有效、中值不行？

> 高斯噪声零均值且每个像素都独立抖动，**多次平均能让正负偏差互相抵消**（大数定律），所以均值有效。
> 中值只是"挑一个值"，没有平均抵消效应，且会丢失真实细节。

Q：为什么中值滤波去椒盐噪声有效、均值不行？

> 椒盐是极端值（0/255），均值会把极端值"分摊"到邻域，污染一片；中值排序后极值在两端，中位数直接忽略它们。

Q：窗口大小（3x3、5x5）怎么影响?

> 窗口越大，去噪越强，但**模糊越严重**、细节损失越多。中值滤波 3x3 常保细节更好，5x5 去噪更狠但更糊。需要**权衡**。本实验 `median_k3` (21.97dB) 优于 `median_k5` (20.86dB) 就是这个道理。

Q：什么是"保边去噪"?

> 在平滑噪声的同时**保持边缘清晰**。中值滤波、双边滤波、引导滤波都属于这类。双边滤波还额外考虑**灰度相似性**（只平均灰度相近的像素，避免跨边缘平均）。

Q：椒盐噪声的"盐"和"椒"分别指什么?

> 盐 = 白点（255，像撒了盐），椒 = 黑点（0，像撒了胡椒粉）。

Q：如果既有高斯又有椒盐噪声怎么办?

> 用**修正阿尔法均值滤波**（排序截尾+平均），或先中值去椒盐再均值/高斯去高斯（级联）。本实验验证了阿尔法均值在混合噪声下最优。

实验四：图像锐化与边缘检测 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**让模糊的图像"变清晰"，并把物体的轮廓（边缘）勾出来。**

去噪（实验三）是"平滑 = 去掉高频"，锐化则**正好相反——增强高频（细节、边缘）**。它俩是互逆操作。

---

2. 为什么"微分"能锐化？（数学根基，必考）

先建立直觉： - **平坦区域**（灰度不变）→ 导数为 0 → 无反应 - **边缘/细节**（灰度突变）→ 导数**很大** → 强响应

所以：**微分（求导）能"检测"灰度变化剧烈的地方 = 边缘**。把微分结果加回原图，就"强调了边缘" = 锐化。

对应到离散图像，求导变成**差分**：

一阶差分（梯度）：

$$\frac{\partial f}{\partial x} = f(x+1, y) - f(x, y)$$

二阶差分（拉普拉斯）：

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

离散形式：

$$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

答辩常问：**为什么锐化用微分/差分而不是别的？**

> 因为边缘的本质就是**灰度的突变**，而微分正是衡量"变化率"的数学工具。平坦区导数为 0，边缘处导数出现极值，天然适合检测边缘。这是所有锐化/边缘算子的共同理论基础。

---

3. 拉普拉斯锐化（二阶微分）

3.1 拉普拉斯算子（模板）

把上面的二阶差分公式写成卷积核：

4 邻域模板（本实验默认）：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

8 邻域模板（更敏感）：

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

规律：中心为正/负，周围为负/正，总和为 0（保证平坦区响应为 0）。

> 注意：上面写的是"周围 1、中心 -4"。有的教材写成"周围 -1、中心 4"，**符号相反**，相应锐化公式的加减号也要反过来。本代码用的是**中心为负（-4/-8）** 的版本。

3.2 锐化公式（把边缘加回去）

$$g(x,y) = f(x,y) + c \cdot [-\nabla^2 f(x,y)]$$

或用卷积核直接一步完成（把拉普拉斯核嵌入单位核）：

$$g = f + c \cdot (f * \mathrm{Lap}) = f * (I + c \cdot \mathrm{Lap})$$

其中 c = 锐化强度系数（$c = 1$ 即标准拉普拉斯锐化）。

大白话：**原图 + 拉普拉斯响应 = 边缘被突出 = 变清晰**。

答辩常问：**拉普拉斯锐化为什么是"原图减拉普拉斯"还是"加"?**

> 取决于模板符号约定。若模板**中心为负**（周围正，如本代码 -4），则 $\nabla^2 f$ 在边缘处为负，用 $f - \nabla^2 f$（即减去一个负数=加）或写成 $f + c \cdot (-\nabla^2 f)$；若模板**中心为正**，则用 $f + \nabla^2 f$。**本质都是让边缘处灰度差被放大**。答辩时说明自己用的模板符号即可。

答辩常问：**4 邻域和 8 邻域模板区别?**

> 4 邻域只考虑上下左右，对**水平/垂直边缘**敏感，对对角边缘响应弱；8 邻域加上四个对角，是**各向同性**的（各个方向一致），对斜边也敏感，但**对噪声更敏感**（参与像素更多）。

答辩常问：**拉普拉斯（二阶）相比一阶梯度的优缺点?**

> 优点：各向同性、能产生**双边缘**（零交叉）、对细线和孤立点响应强、计算简单； 缺点：**对噪声极其敏感**（二阶微分放大噪声更严重）、会产生双像素宽边缘、丢失边缘方向信息。

---

4. 非锐化掩蔽 Unsharp Masking（USM）

4.1 原理

$$g = f + k \cdot (f - \bar{f})$$

其中： - $\bar{f}$ = 原图的**模糊版本**（低通/高斯平滑结果） - $f - \bar{f}$ = **高频细节（边缘）** 分量 - k = 锐化强度（本实验 $k = 1.5$，叫 amount）

大白话：**先把图模糊一份，用原图减去模糊版 = 得到"清晰的边缘部分"，再把这个边缘按倍数加回原图。**

代码：`unsharp_mask(ksize=5, amount=1.5)`，用高斯模糊得 $\bar{f}$。

答辩常问：**USM 和拉普拉斯锐化什么关系?**

> 本质**完全一样**！都是"原图 + 高频分量"。区别在**高频怎么来**： - 拉普拉斯：用**二阶微分算子**提取高频 - USM：用**原图减模糊版**（低通的反面）提取高频 二者殊途同归，USM 因为用可调高斯模糊，**更灵活、噪声更可控**，所以 Photoshop 等软件都用 USM。

答辩常问：**为什么叫"非锐化掩蔽"?**

> 历史名词。早期印刷工艺中，用一张"虚化（非锐化）的负片"作为掩模（mask）叠在原片上，从而增强边缘。名字沿用至今。

---

5. Canny 边缘检测（五步法，超高频考点）

Canny 是**最优边缘检测**的经典算法，不是简单一个算子，而是**一套流程**：

步骤 1：高斯平滑去噪

先用高斯滤波平滑，因为后续求导会放大噪声。

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

步骤 2：计算梯度（强度和方向）

用 Sobel 等算子求 G_x、G_y：

$$|\nabla f|=\sqrt{G_x^2+G_y^2},\quad \theta=\arctan\left(\frac{G_y}{G_x}\right)$$

梯度**幅值** = 边缘强度，**方向** = 边缘法向。

步骤 3：非极大值抑制（NMS）——细化边缘

只保留梯度方向上的局部最大值，其余置 0。

效果：把"粗边（几个像素宽）"**细化成单像素宽的细边**。

大白话：沿着边缘的垂直方向比较，只有"**最突出**"的那个点留下，旁边的"次突出"被抑制掉。

步骤 4：双阈值检测（本实验重点）

用**两个阈值** T_{low}、T_{high}（本实验测了 50-100、100-200 等组合）：

- 梯度 > T_{high} → **强边缘**，确定保留
- 梯度 < T_{low} → **抑制**，确定丢弃
- T_{low} ≤ 梯度 ≤ T_{high} → **弱边缘，看它是否与强边缘相连**：连着则保留，否则丢弃

答辩常问：**为什么要双阈值，一个阈值不行吗?**

> 单阈值**两难**：阈值高了会**断边**（真实边缘丢失、不连续），阈值低了会**引入大量假边缘**（噪声响应）。双阈值用"高阈值定真骨架 + 低阈值补连接"，**既保连续又压噪声**，是 Canny 的精髓。

答辩常问：**本实验不同双阈值的效果差异?**

> 阈值 50-100：保留边缘**最多最完整**（连弱边也留），但可能有少量噪声边；阈值 100-200：**只剩强边缘**，轮廓干净但可能断裂、丢失细节。**阈值越高，边缘越少越干净**。

步骤 5：滞后边界跟踪

就是步骤 4 中"弱边缘连接强边缘"的过程，通过**连通性**判断把断断续续的边缘串起来。

Canny **三大评价标准（记住）**： 1. **低错误率**：尽可能少地把非边缘判为边缘、少漏检真边缘 2. **高定位性**：检测到的边缘位置尽可能接近真实边缘 3. **单一响应**：一条真实边缘只产生**一个**响应（不出现多像素宽或重复）

---

6. 本实验代码对照

理论	代码位置	实现
拉普拉斯 4/8 邻域	`laplacian_sharpen(kernel_type="4"/"8")`	`cv2.filter2D` + 自定义核
锐化 $g = f + c(-\nabla^2 f)$	同上	原图减拉普拉斯响应
USM $g = f + k(f - \bar{f})$	`unsharp_mask(ksize=5, amount=1.5)`	高斯模糊后加权
Canny 双阈值	`canny_edges(t1,t2)`	`cv2.Canny`

运行：

```
python sharpening.py
python sharpening.py ../../../../data/standard/cameraman.png
```

（cameraman 边缘清晰，特别适合本实验）

---

7. 答辩高频问题速答

Q：锐化和去噪（平滑）的关系?

> **互逆**。平滑 = 低通 = 衰减高频 = 去噪但模糊；锐化 = 高通 = 增强高频 = 突出边缘但放大噪声。两者是一对矛盾，实际处理要**权衡**（保边去噪就是要兼顾）。

Q：为什么锐化会放大噪声?

> 因为噪声本身就是**高频信号**（像素级随机跳变），锐化增强高频时**不加区分地把噪声也一起放大了**。这就是为什么锐化前常常先做一次轻度平滑，或 Canny 第一步就是高斯去噪。

Q：一阶微分和二阶微分检测边缘的区别?

> 一阶（梯度）：边缘处产生**单峰（极值）**，能给出边缘**方向**，对噪声相对鲁棒，但边缘较粗； 二阶（拉普拉斯）：边缘处产生**零交叉**，各向同性，对细节（细线、孤立点）响应更强，但**对噪声极敏感**，且丢失方向信息。

Q：Sobel 和 Canny 区别?

> Sobel 只是**一个梯度算子**（步骤 2 那一步），输出粗的梯度图，需要自己再阈值化；Canny 是**完整流程**（平滑+梯度+NMS+双阈值+跟踪），输出**单像素宽、连续、干净**的二值边缘图，质量高得多。

Q：非极大值抑制 NMS 的作用?

> 把梯度幅值在**边缘法线方向**上不是局部极大的点置零，从而把宽边**细化成一个像素宽**。没有 NMS，Canny 输出的边会是好几像素宽、模糊的。

Q：为什么 Canny 要先平滑?

> 因为梯度/微分运算会**放大噪声**，不先平滑的话，噪声会产生大量虚假边缘。高斯平滑是 Canny"低错误率"目标的保障。

实验五：频率域滤波 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**把图像从"看像素"的世界，搬到"看频率"的世界，在那里把噪声/模糊滤掉，再搬回来。**

前面四个实验都在**空间域**（直接对像素操作）。本实验换一个视角：**频率域**——把图像看成"由不同频率的波叠加而成"。

---

2. 傅里叶变换：图像 = 一堆波的叠加（核心直觉）

2.1 一维连续傅里叶变换

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi ux} dx$$

逆变换：

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{j2\pi ux} du$$

2.2 二维离散傅里叶变换 DFT（图像用的）

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cdot e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

逆变换 IDFT：

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) \cdot e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

代码用 FFT（**快速傅里叶变换**） 加速：`np.fft.fft2`。

2.3 频率到底是什么意思？（讲人话）

把图像想成地形： - **低频** = 大范围的平缓起伏（大片天空、墙面、背景）→ 对应图像中**变化慢的区域** - **高频** = 剧烈的局部抖动（边缘、纹理、细节、噪声）→ 对应图像中**突变的地方**

- $F(0,0)$（**直流分量** DC）= 整幅图的**平均亮度**
- 离中心越远 = 频率越高 = 越细的细节

所以：低通 = 保留背景、模糊细节（去噪）；高通 = 保留边缘、去掉背景（锐化）。

答辩常问：**频率域的低频/高频对应图像里的什么？**

> 低频 = 灰度缓慢变化的大片平坦区域（背景、主体轮廓）；高频 = 灰度快速变化的部分（边缘、纹理、细节、噪声）。

答辩常问：F(0,0) 是什么？为什么频谱图中心最亮？

> $F(0,0)$ 是直流分量，等于全图平均亮度乘像素数，通常是**最大的值**。做 `fftshift` 中心化后它跑到图像中心，所以**中心最亮**。

---

3. 频谱图怎么看 & 为什么要中心化和取 log

3.1 中心化 fftshift

原始 FFT 输出的低频在**四角**，高频在中心，不直观。用：

$$F_{shift}(u,v) = F\left(u - \frac{M}{2}, v - \frac{N}{2}\right)$$

即 `np.fft.fftshift`，把**低频搬到图像中心**，向外频率递增，符合直觉。

3.2 取对数压缩（呼应实验二）

频谱动态范围极大（$|F|$ 可能从 0 到 10^6），直接显示几乎全黑。所以：

$$S(u,v) = \log\left(1 + |F(u,v)|\right)$$

取对数后压缩大值、提升小值，才能看清完整结构（**这就是实验二对数变换的实际应用，答辩可以联动回答！**）。

3.3 频谱里的"十字亮带"是什么？

本实验 baboon 图的频谱能看到**水平和垂直的亮线**。

原因：图像边界/结构中存在**强水平、垂直方向的边缘**，这些方向性的突变在频域表现为**对应方向的能量集中**。另外，**图像自身的矩形边界（非周期截断）**也会产生十字状频谱泄漏。

答辩常问：**频谱图中的亮线/十字代表什么？**

> 代表图像中存在**该方向的强周期性结构或直边缘**。水平亮线对应垂直方向的边缘/条纹，垂直亮线对应水平方向的边缘/条纹。若是规则纹理（如布料、栅格），频谱会出现**规则排列的亮点**。

---

4. 卷积定理：为什么频域滤波更快？（必考）

空间域卷积 = 频域乘积：

$$f(x,y) * h(x,y) \iff F(u,v) \cdot H(u,v)$$

反之：

$$f(x,y)\cdot h(x,y) \iff F(u,v) * H(u,v)$$

意义：空间域做卷积（复杂度高），等价于频域**逐点相乘**（复杂度低）。所以大核滤波在频域做更快。

本实验滤波流程（`apply_filter`）：1. `fft2` 到频域 → F 2. `fftshift` 中心化 3. 乘以滤波器 H：$G = F \cdot H$ 4. `ifftshift` 反中心化 5. `ifft2` 回到空间域

答辩常问：**频域滤波和空间域滤波（如实验三中值）等价吗？**

> **线性滤波**（均值、高斯）在频域和空间域**完全等价**（由卷积定理保证），只是实现方式不同。但**非线性滤波**（中值、最大值）**没有对应的频域表达**，只能在空间域做。这是频域方法的局限。

---

5. 三类滤波器对比（核心）

设 $D(u,v)$ = 频域中某点到中心的距离：

$$D(u,v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

D_0 = **截止频率**（决定保留多大半径内的频率）。

5.1 理想低通滤波器 ILPF

$$H(u,v) = \begin{cases} 1 & D(u,v) \le D_0 \\ 0 & D(u,v) > D_0 \end{cases}$$

- 特点：**一刀切**，半径内全通过，外全砍掉
- 缺点：产生严重**振铃（ringing）** 现象（图像边缘出现同心波纹）
- 原因：频域的**矩形（阶跃）截断** = 空间域与 sinc **函数**卷积，sinc 有振荡旁瓣 → 振铃

代码：`low = (dist <= d0)`

5.2 巴特沃斯低通滤波器 BLPF（n 阶）

$$H(u,v) = \frac{1}{1 + \left[\frac{D(u,v)}{D_0}\right]^{2n}}$$

- n = 阶数（本实验 `order=2`）
- 特点：**平滑过渡**，无尖锐截断
- 优点：n 越大越接近理想（但仍平滑），**振铃很轻**
- $H(D_0) = 0.5$，即截止频率处衰减到一半（-3dB **点**）

代码：`low = 1.0 / (1.0 + (dist / d0) ** (2 * order))`

5.3 高斯低通滤波器 GLPF

$$H(u,v) = e^{-\frac{D^2(u,v)}{2D_0^2}}$$

- 特点：**最平滑**的过渡，高斯形状

- 优点：**完全无振铃**（因为高斯的傅里叶逆变换还是高斯，没有振荡旁瓣）
- 缺点：过渡带最宽，截止不够"干脆"

代码：`low = np.exp(-(dist ** 2) / (2 * d0 ** 2))`

5.4 高通 = 1 - 低通（本实验做法）

$$H_{high}(u,v) = 1 - H_{low}(u,v)$$

代码：`return low, 1.0 - low` —— 直接一步生成低通和高通。

高通效果：保留边缘和细节，**去掉平缓背景**，图像整体变暗（因为去掉了直流/低频），呈现"只有轮廓"的效果。

5.5 三类对比总结（背下来）

滤波器	过渡带	振铃	平滑度	模糊程度
理想	无（陡峭）	**严重**	差	中
巴特沃斯	中等	轻微（n 大增）	中	中
高斯	最宽	**无**	最好	最平滑

答辩常问：**为什么理想低通有振铃而高斯没有？**

> 频域的**锐截止（矩形窗）** 对应空间域的 sinc **函数**，sinc 有正负交替的旁瓣，与图像卷积就产生同心波纹（振铃），这是**吉布斯效应**。高斯的傅里叶逆变换仍是高斯，**单调无振荡**，所以完全无振铃。

答辩常问：**截止频率 D0 怎么影响结果？**

> D_0 越小 → 保留频率越少 → **越模糊**（低通）；D_0 越大 → 保留越多 → 越接近原图。高通反之：D_0 越小保留的边缘越多。本实验 `butterworth_d0_compare.png` 展示了不同 D_0 的对比。

---

6. 本实验代码对照

理论	代码位置	实现
FFT + 中心化	`fft_spectrum()`	`np.fft.fft2` + `fftshift`
对数压缩频谱	同上	$\log(1+\|F\|)$
三类滤波器	`make_filters()`	用 `dist` 构造 low/high
频域滤波流程	`apply_filter()`	$F \cdot H$ → `ifftshift` → `ifft2`

运行：

```
python frequency_filter.py
python frequency_filter.py ../../../../data/standard/baboon.jpg
```

（baboon 纹理密集，频谱结构最丰富，效果最明显）

---

7. 答辩高频问题速答

Q：为什么要用频率域？空间域不够吗？

1. 有些操作在频域**更直观**（如去除周期性条纹噪声，频谱上就是几个亮点，去掉即可）；
2. 大核卷积在频域**计算更快**（卷积定理，复杂度从 $O(N^2K^2)$ 降到 $O(N^2 \log N)$）；
3. 但**非线性滤波**（中值等）只能在空间域做，这是频域的局限。

Q：FFT 后得到的是复数，怎么处理？

FFT 结果是复数，包含**幅度谱** $|F| = \sqrt{Re^2 + Im^2}$ 和**相位谱** $\phi = \arctan(Im/Re)$。滤波时对**复数整体**操作（保留相位），逆变换后取**实部/模**得到图像。相位谱非常重要——**保留相位+交换幅度**的实验表明，相位决定图像的结构信息。

Q：低通滤波在图像上是什么效果？高通呢？

低通 = **模糊/平滑/去噪**（保留背景，去掉细节噪声）；高通 = **锐化/边缘提取**（保留边缘细节，去掉平缓背景，图像整体变暗）。

Q：振铃是怎么产生的？怎么消除？

由频域**陡峭截断**（理想滤波器）引起的吉布斯效应，空间域表现为边缘附近的同心波纹。消除方法：**改用平滑过渡的滤波器**（巴特沃斯、高斯），或对理想滤波器加**窗函数**软化边界。

Q：频域滤波后的图像为什么要用 np.abs？

逆变换 `ifft2` 结果理论上应为实数，但因浮点误差会残留极小虚部，取 `np.abs`（模）得到实数灰度值，再 clip 到 [0,255]。

Q：本实验高通后图像整体偏暗，为什么？

因为高通滤除了**低频/直流分量**，而直流代表**平均亮度**。去掉后图像均值大幅下降（趋向 0），所以整体变黑，只剩亮的边缘。常用做法是**高频提升**：$H_{high} = 1 + k \cdot H_{highpass}$，保留一部分低频。

实验六：图像复原 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**照片拍糊了，用数学方法"反推"出原来清晰的图。**

注意它和"图像增强"（实验一~五）的**本质区别**： - **增强（Enhancement）**：主观的，让图"看起来更好"，不管原来是什么 - **复原（Restoration）**：客观的，先**建模"图像是怎么变糊的"**，然后**数学求逆**，尽量恢复**原始真实图像**

本实验：人为制造模糊（高斯模糊、运动模糊）→ 加噪 → 用**逆滤波**和**维纳滤波**复原 → 用 PSNR 比较。

---

2. 退化模型（一切的基础，必考）

图像退化被建模为：

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$$

其中： - $f(x, y)$ = **原始清晰图**（未知，要求的目标） - $h(x, y)$ = **退化函数 / 点扩散函数 PSF**（模糊的"原因"，如失焦、运动） - $*$ = 卷积 - $\eta(x, y)$ = **加性噪声** - $g(x, y)$ = **观察到的模糊图**（已知）

频域形式（关键，由卷积定理）：

$$G(u, v) = H(u, v) \cdot F(u, v) + N(u, v)$$

其中大写字母是对应的傅里叶变换：$G = \mathrm{F}\{g\}$, $H = \mathrm{F}\{h\}$, $F = \mathrm{F}\{f\}$, $N = \mathrm{F}\{\eta\}$。

大白话：**模糊 = 清晰图 卷积 一个"糊化核" PSF，再加噪声。** 复原就是已知 G 和 H，求 F。

答辩常问：**图像复原和图像增强的区别？**

> 增强是**主观**的（让图更好看，如对比度拉伸、锐化），不需要知道退化过程；复原是**客观**的，必须**先建立退化模型**（知道/估计 PSF 和噪声），再用数学逆运算**逼近原始真实图像**。复原是"还原真相"，增强是"美化观感"。

---

3. PSF 点扩散函数（模糊核）

PSF（Point Spread Function）：一个理想点光源经过成像系统后"扩散"成的样子。**它就是上式中的 h。**

3.1 高斯模糊 PSF（模拟失焦/镜头模糊）

二维高斯函数：

$$h(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$$

σ 越大 → 扩散越宽 → **越模糊**。

代码做法很巧妙：先建一个只有中心为 1 的脉冲（delta），对它做高斯模糊，得到的就是高斯 PSF；再用 `filter2D` 与原图卷积。

```
psf[ksize//2, ksize//2] = 1.0           # 脉冲
psf = cv2.GaussianBlur(psf, (k,k), sigma)  # 得到高斯 PSF
blurred = cv2.filter2D(img, -1, psf)       # 卷积 = 模糊
```

3.2 运动模糊 PSF（模拟相机/物体运动）

物体与相机在曝光期间有**相对匀速直线运动**，等效于沿运动方向把图像"拖"出一条线：

$$h(x,y)=\begin{cases}\frac{1}{L} & \text{在长度为 } L\text{、角度为 } \theta \text{ 的线段上}\\ 0 & \text{其他}\end{cases}$$

参数：`length`（运动长度）、`angle`（运动角度）。

代码：在 `length×length` 的核里沿角度方向画一条线，归一化（除以和，保证能量守恒、整体亮度不变），再卷积。

答辩常问：**为什么 PSF 要归一化（psf /= psf.sum()）？**

> 保证滤波器的**直流增益为 1**，即卷积后图像的整体平均亮度不变（能量守恒）。若不归一化，图像会整体变亮或变暗，不符合"只是变糊"的物理退化模型。

---

4. 逆滤波（Inverse Filtering）—— 最直接但最脆弱

4.1 原理

忽略噪声（$N=0$），频域退化式变为 $G=H\cdot F$，直接除：

$$\hat{F}(u,v)=\frac{G(u,v)}{H(u,v)}$$

然后逆变换：$\hat{f}=\mathcal{F}^{-1}\{\hat{F}\}$。

看起来完美，问题在哪？

4.2 致命弱点（必考）

1. **$H(u,v)$ 接近 0 时爆炸**：实际中 H 在高频处往往很小甚至为 0，此时 $G/H\to$ 无穷大，结果完全被噪声支配
2. **忽略噪声**：真实情况 $G=HF+N$，逆滤波得到：

$$\hat{F}=\frac{HF+N}{H}=F+\frac{N}{H}$$

噪声项 N/H 在 H 小的地方被极度放大！

本实验实测：加噪后逆滤波 PSNR 仅 5.60~6.94 dB，几乎不可用 —— 完美印证了这个理论缺陷。

代码做了**截断保护**（但仍不够）：

```
h_safe = np.where(np.abs(h) < eps, eps, h)  # 小于eps的置为eps
restored = np.fft.ifft2(g / h_safe)
```

答辩常问：**逆滤波为什么在实际中几乎不能用?**

> 因为它**完全没有考虑噪声**。真实退化含噪声 N，逆滤波结果为 $F + N/H$，在 $H(u,v)$ 接近零的高频处，噪声被**无限放大**，复原图布满噪声、完全失效。只有在**无噪声且 H 无零点**的理想情况下才有效。

---

5. 维纳滤波（Wiener Filtering）—— 本实验的主角

5.1 原理：最小均方误差（MMSE）最优

维纳滤波不再"硬除"，而是找一个滤波器，使复原图 $\hat{f}$ 与真实图 f 的**均方误差最小**：

$$\min_{\hat{f}} \; E[\|f - \hat{f}\|^2]$$

求解这个优化问题，得到维纳滤波器：

$$\boxed{\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2 + K}\right] G(u,v)}$$

其中： - $H^*(u,v)$ = H 的**复共轭** - $|H(u,v)|^2 = H \cdot H^*$ - K = **噪声对信号的功率比**（常数近似），$K = \frac{S_\eta(u,v)}{S_f(u,v)}$

代码（完全对应公式）：

```
h_conj = np.conj(h)
restored = np.fft.ifft2(h_conj / (np.abs(h)**2 + k) * g)
```

5.2 为什么维纳滤波更强？（对比逆滤波）

把维纳公式和逆滤波 $\hat{F} = G/H$ 比较：

当 **无噪声**（$K = 0$）时：

$$\hat{F} = \frac{H^*}{|H|^2} G = \frac{H^*}{HH^*} G = \frac{G}{H}$$

维纳退化为逆滤波！ 说明逆滤波是维纳在无噪情况下的特例。

当 **有噪声**（$K > 0$）时： - 分母多了 $+K$，**避免了除零** - 在 $|H|^2 \ll K$（H 很小）的地方，$\hat{F} \approx \frac{H^*}{K} G \approx 0 \rightarrow$ **自动抑制**这些不可靠的高频，**不让噪声放大** - 在 $|H|^2 \gg K$（H 大）的地方，$\hat{F} \approx G/H \rightarrow$ 接近逆滤波，正常复原

大白话：维纳滤波**聪明地知道"哪里可信"**——在信号强于噪声的频率正常复原，在噪声强于信号的频率**自动"认怂"不放大**，这是它在有噪时远好于逆滤波的原因。

本实验实测：加噪后维纳 PSNR 14.31~15.22 dB，虽不完美但远优于逆滤波的 5~7 dB。

5.3 参数 K 的影响（本实验做了 0.001 / 0.01 / 0.1 对比）

- K **太小** → 接近逆滤波，去模糊强但**噪声大**（残余噪声明显）
- K **太大** → 过度平滑，**去模糊不足**（图像仍糊），但噪声小
- 需要**权衡**，本实验默认 $K = 0.01$

答辩常问：**K 值怎么选?**

> K 是噪声功率与信号功率之比。理论上应逐频率取 S_η/S_f，实际中常**整图取一个常数**（近似），通过实验调节： - 噪声大 → 增大 K（更保守、更平滑） - 噪声小 → 减小 K（更接近逆滤波、更锐利） 也可根据**噪声方差/图像方差**估计。本实验通过对比 0.001/0.01/0.1 展示了这个权衡。

---

6. 逆滤波 vs 维纳滤波 总结（背下来）

	逆滤波	维纳滤波
是否考虑噪声	**否**	**是**（MMSE 最优）
无噪时	有效	**退化为逆滤波**
有噪时	**失效**（PSNR 5~7dB）	**稳健**（PSNR 14~15dB）
$H \approx 0$ 处	爆炸	被 K 抑制，趋于 0
需要参数	只需 H	需 H 和 K（噪声功率比）
本质	直接求逆	**带正则化的最优估计**

---

7. 本实验代码对照

理论	代码位置	实现
高斯 PSF	`gaussian_blur(ksize=15, sigma=3.0)`	脉冲+GaussianBlur
运动 PSF	`motion_blur(length=20, angle=0)`	沿角度画线并归一化
PSF 补齐尺寸	`pad_psf()`	补零到图像大小
逆滤波	`inverse_filter(eps=1e-3)`	G/H，小值截断
维纳滤波	`wiener_filter(k=0.01)`	$H^*/(\|H\|^2 + K) \cdot G$
PSNR 评价	`psnr()`	同实验三
K 值对比	主流程 `for kk in (0.001,0.01,0.1)`	展示权衡

运行：

```
python restoration.py
python restoration.py ../../../../data/standard/sudoku.png
```

（sudoku 有文字数字，复原后文字可读性一目了然）

---

8. 答辩高频问题速答

Q：图像复原和图像增强的核心区别？（再强调，最高频）

增强是**主观/经验**的，无模型，目标"看起来更好"；复原是**客观/模型驱动**的，需建立退化模型 $g = h * f + \eta$，通过数学逆运算**恢复原始真实图像**，通常用 PSNR 等量化指标评价。

Q：为什么逆滤波在噪声下失效，而维纳不会？

逆滤波 $\hat{F} = F + N/H$，在 $H \approx 0$ 处噪声被无穷放大。维纳在分母加 K，使这些频率的响应**自动衰减到 0**，抑制噪声，是有约束的最优估计。

Q：如果不知道 PSF（模糊核）怎么办？

这叫**盲去卷积（Blind Deconvolution）** 或 PSF **估计**。方法包括：观察模糊图像中边缘/点源的扩散形状估计、用参数化模型（已知是运动模糊则估角度和长度）、或用深度学习直接端到端去模糊。本实验是**非盲**复原（已知 PSF）。

Q：维纳滤波的 K 能不能自适应？

理论最优是逐频率的 $K(u,v) = S_\eta(u,v)/S_f(u,v)$，需要知道噪声和信号的功率谱。实际常用常数近似，或用**迭代维纳**、或先用噪声估计（如从平坦区估 σ_n^2）来设定 K。

Q：为什么复原结果普遍 PSNR 不算特别高（十几 dB）？

因为**模糊是不可逆的信息损失**（高频被 H 压到接近 0，且叠加了噪声），复原是**病态逆问题（ill-posed）**，只能"最优估计"而非"完美还原"。模糊越重、噪声越大，复原上限越低。这是信息论层面的固有限制。

Q：什么是病态逆问题？

解**不唯一或不稳定**（输入微小扰动导致输出巨大变化）。图像复原中，多个不同的 f 可能产生几乎相同的 g，且 H 近零处放大噪声，所以必须加**正则化**（维纳的 K 就是一种正则化）来稳定求解。

Q：高斯模糊和运动模糊的 PSF 本质区别？

高斯模糊是**各向同性**的（各个方向对称扩散，模拟失焦/散焦）；运动模糊是**各向异性**的（只沿运动方向有线状扩散，有明确方向性），所以运动模糊的频谱会出现**平行的暗条纹**（PSF 的零点）。

实验七：肤色分割 —— 底层原理

> 目标：讲人话，让小白也能在答辩中对答如流。

1. 这个实验到底在干什么？

一句话：**在一张彩色照片里，把"皮肤"的像素找出来，其他全部屏蔽掉（做成黑白掩膜）。**

这是典型的**彩色图像分割**问题，也是人脸检测、手势识别、敏感内容过滤的第一步。

本实验用**三种颜色空间**分别做肤色检测，再**投票融合**，并用真实证件照测试。

---

2. 为什么不能直接用 RGB？（核心问题，必考）

RGB（红绿蓝）三个分量**高度相关于亮度**。

问题：同一个人的皮肤，在**强光下** RGB 值都很大，在**阴影里**都很小，但**它都是皮肤**。反之，一张白纸在暗处可能和皮肤在亮处的 RGB 完全一样。

结论：RGB 把"颜色"和"亮度"混在一起，无法稳定描述肤色。

所以需要**把亮度（Luminance）和色度（Chrominance）分离**的颜色空间： - YCbCr / YUV：Y = 亮度，Cb/Cr = 蓝色差/红色差（色度） - HSV / HSI：H = 色调（颜色种类），S = 饱和度（鲜艳程度），V = 明度（亮度）

关键优势：在这些空间里，**肤色的色度聚集在一个很小的范围**，且与亮度基本解耦。我们只需对色度设阈值，就能鲁棒地检测肤色（不受明暗影响）。

答辩常问：**为什么肤色检测要用 YCbCr/HSV 而不用 RGB？**

> 因为 RGB 三分量都与亮度强相关，肤色在 RGB 空间分布**分散且随光照剧烈变化**；而 YCbCr/HSV 将**亮度与色度分离**，肤色在色度平面（Cb-Cr 或 H-S）上**聚集在紧凑的小区域**，且对光照变化鲁棒，便于用简单阈值分割。

---

3. 三种颜色空间与肤色模型

3.1 RGB 空间（规则法 —— 最朴素）

用一组**经验不等式**描述"肤色"的 RGB 特征（归一化到 0~255）：

$$\begin{cases} R > 95,\ G > 40,\ B > 20 \\ \max(R,G,B) - \min(R,G,B) > 15 \\ |R - G| > 15 \\ R > G,\ R > B \end{cases}$$

每一条的含义（讲人话）： - $R>95, G>40, B>20$：排除**太暗**的像素（暗处不判断） - $\max-\min>15$：排除**接近灰度**的像素（白/灰/黑墙，因为皮肤有颜色，不是纯灰） - $|R-G|>15$：排除 R 和 G 太接近的（排除偏绿/黄绿） - $R>G$ 且 $R>B$：**皮肤偏红**，红分量必须最大

缺点：受光照影响极大。本实验实测：RGB **对光照最敏感，阴影处容易漏检**。

代码：

```
mx = np.maximum(np.maximum(r,g), b); mn = np.minimum(np.minimum(r,g), b)
mask = ((r>95)&(g>40)&(b>20)&((mx-mn)>15)&(np.abs(r-g)>15)&(r>g)&(r>b))
```

3.2 YCbCr 空间（最经典，推荐）

转换（从 RGB，数字视频标准 BT.601）：

$$Y = 0.299R + 0.587G + 0.114B$$

$$Cb = -0.169R - 0.331G + 0.500B + 128$$

$$Cr = 0.500R - 0.419G - 0.081B + 128$$

- Y = 亮度（luma）
- Cb = 蓝色分量与亮度的差（Blue-difference）
- Cr = 红色分量与亮度的差（Red-difference）

肤色范围（经典经验区间，本实验采用）：

$$77 \le Cb \le 127 \quad 且 \quad 133 \le Cr \le 173$$

为什么有效：不同人种、不同光照下的皮肤，其 Cr-Cb **色度值惊人地集中**在这个小矩形区域内（这就是著名的"肤色聚类"现象）。而 Y（亮度）**不设阈值**，所以对明暗鲁棒。

代码：`cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)`，再 `cv2.inRange`。

答辩常问：**为什么 YCbCr 里对 Y 不设限?**

> 因为肤色的**色度（Cb、Cr）与亮度 Y 基本独立**。设 Y 阈值反而会误伤暗处/亮处的皮肤。只对 Cb、Cr 设限就能在不同光照下鲁棒检测肤色，这正是色度空间分离的优势。

3.3 HSV 空间（本实验实测最干净）

- H（Hue **色调**）：颜色种类（红橙黄绿青蓝紫），OpenCV 中范围 0~179（不是 0~360，除以 2 存进 8 位）
- S（Saturation **饱和度**）：颜色纯度/鲜艳度，0~255
- V（Value **明度**）：亮度，0~255

肤色范围（本实验采用，注意 OpenCV 的 H 是 0~179）：

$$H \in [0, 25], \quad S \in [30, 170], \quad V \ge 50$$

含义： - $H \leq 25$：皮肤色调偏**红/橙**（对应 OpenCV 中 0~25，即真实角度 0°~50°） - $S \geq 30$：排除接近灰白的低饱和区域（白墙、白纸） - $S \leq 170$：排除过度饱和（纯红等异常） - $V \geq 50$：排除太暗的噪声

本实验实测：HSV 在蓝底、白底证件照上分割**最干净**，肤色区域完整、背景残留少。

答辩常问：OpenCV 里 H 范围为什么是 0~179 而不是 0~360?

> 因为 OpenCV 用 **8 位无符号整数（0~255）** 存储，为了把 0~360 的色调角存进去，将其**除以 2** 压缩到 0~179。所以设阈值时要注意换算：真实角度 ×2 = OpenCV 的 H 值（如真实 50° → OpenCV 的 25）。

答辩常问：YCbCr 和 HSV 哪个更好?

> 各有优劣：YCbCr 计算简单（线性变换，快），肤色聚类紧，但对**亮度剧烈变化**仍有一定敏感；HSV 更符合人类直觉，H 通道对光照鲁棒性更好，实测在证件照上更干净。实际常**融合多个空间**使用（本实验的投票融合就是这个思路）。

---

4. 形态学后处理（去噪点、填空洞）

阈值分割后，掩膜上会有**孤立小白点（噪声误检）** 和**皮肤区内的小黑洞（漏检）**。用**形态学**修复：

4.1 开运算（Opening）—— 去小白点

$$A \circ B = (A \ominus B) \oplus B$$

先腐蚀再膨胀。作用：去掉**比结构元素小的孤立亮点**，平滑物体边界，断开细连接。

4.2 闭运算（Closing）—— 填小黑洞

$$A \bullet B = (A \oplus B) \ominus B$$

先膨胀再腐蚀。作用：填充**比结构元素小的暗孔洞**，连接邻近区域，平滑边界。

本实验用 `MORPH_ELLIPSE` 5×5 结构元素，先开后闭：

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)   # 去噪点
cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, kernel) # 填孔洞
```

为什么用椭圆结构元素? 因为人脸/皮肤区域接近圆形，椭圆核**各向同性**，不会引入方向性偏差。

答辩常问：**开运算和闭运算分别解决什么问题?**

> 开运算（先腐蚀后膨胀）**去除孤立亮点/毛刺**，消除噪声误检；闭运算（先膨胀后腐蚀）**填充内部小孔洞和缝隙**，修复漏检。两者结合让掩膜更干净完整。

4.3 最大连通域（定位人脸）

```
num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(cleaned, 8)
largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
```

找出**面积最大的连通区域**作为人脸候选（背景的零散误检区域面积小，被排除）。

---

5. 投票融合（本实验的亮点）

三种方法各有优缺点，把它们的掩膜**投票**： - 每个像素统计"被多少种方法判为皮肤" - **≥2 票**（多数）才最终判为皮肤

$$M_{final}(x,y) = \begin{cases} 1 & \Sigma_i M_i(x,y) \geq 2 \\ 0 & \text{其他} \end{cases}$$

效果：单一方法的偶然误检（只有 1 票）被否决，**背景误检明显减少**，比任一单一方法都稳定。本实验实测融合结果最干净。

答辩常问：**为什么要做投票融合?**

> 单一颜色空间都有局限（RGB 怕光照、YCbCr 对极端光敏感、HSV 在近灰度区不稳）。融合能**互补**：真正皮肤通常被多个模型同时认可，而背景偶然落入某一个模型区间的概率高、同时落入多个的概率低。投票显著提升**鲁棒性和精度**，降低误检率。

---

6. 本实验代码对照

理论	代码位置	实现
RGB 规则	`skin_mask_rgb()`	一组不等式
YCbCr 范围	`skin_mask_ycbcr()`	77≤Cb≤127, 133≤Cr≤173
HSV 范围	`skin_mask_hsv()`	H≤25, 30≤S≤170, V≥50
开闭运算	`postprocess()`	`MORPH_OPEN` / `CLOSE` + 椭圆核
最大连通域	同上	`connectedComponentsWithStats`
投票融合	主流程	多掩膜求和取 ≥2
批量处理证件照	`main()`	读 `data/raw/` 目录

运行：

```
python skin_segmentation.py
python skin_segmentation.py ../../../../data/raw        # 批量证件照
python skin_segmentation.py ../../../../data/raw/xxx.jpg # 单张
```

7. 答辩高频问题速答

Q：肤色分割属于哪类图像分割方法?

> 属于**基于颜色/阈值的区域分割**（或叫"颜色聚类分割"）。它依据像素在颜色空间的统计分布（聚类）设定边界，而非边缘检测或区域生长。是**像素级（低级）** 分割。

Q：为什么不同人种的肤色能用同一套 CbCr 阈值?

> 研究发现，不同人种肤色的差异主要体现在**亮度** Y上，而在**色度** Cb-Cr **平面**上分布高度重叠、集中在窄区域。这就是肤色检测能用统一色度阈值的生理/物理基础。（但对极端深肤色仍需适当放宽区间。）

Q：背景有类似皮肤的颜色（如棕黄色墙壁）怎么办?

> 这正是单一颜色模型的局限。解决办法： 1. **融合多个颜色空间**（本实验投票） 2. 加入**纹理/形状/位置**等约束 3. 用**统计模型**（如高斯混合模型 GMM、贝叶斯分类器）建模肤色分布，而非硬阈值 4. 用**人脸检测器**（如 Haar/Adaboost、深度学习）先定位人脸区域，再在里面分割肤色

Q：形态学结构元素大小怎么选?

> 取决于要去除的噪声点和要填补的孔洞尺寸。核太小（3×3）去噪不彻底；太大（如 15×15）会**过度腐蚀**导致皮肤区域缩小、细节丢失。本实验用 5×5 椭圆核，在证件照尺度下取得平衡。

Q：为什么实验七用彩色图而前几个实验用灰度图?

> 因为肤色信息**本质在颜色（色度）里**，转灰度会**完全丢失**色度信息，无法区分皮肤和灰度相近的背景。前面的实验（直方图、去噪、锐化、频域）主要处理**亮度/结构**信息，灰度即可；而肤色分割必须**保留彩色信息**。

Q：怎么评价分割效果好坏？（若无真值标注）

> 有真值时可用 IoU（**交并比**）、Dice **系数**、**精确率/召回率**、**像素准确率**；
>
> $$IoU = \frac{|A \cap B|}{|A \cup B|}$$
>
> 无真值时只能**目视检查**或用间接指标（如肤色区域的紧致度、连通域数量）。

Q：如果光照严重偏色（如暖黄灯下）怎么办?

> 可先做**白平衡/颜色恒常性**预处理（如灰度世界假设、完美反射法），校正整体色偏后再分割；或在建模时把光照条件纳入训练集（用统计/学习方法而非固定阈值）。